

Programming Contest for Girls

Example Answers v3.2

Contents

Problem 1 Days in a Leap Year 1
Problem 2 Limited Addition 1
Problem 3 Stickers 1
Problem 4 Tech Support 2
Problem 5 AI Chat-bot 2
Problem 6 Ribbons 2
Problem 7 Alex's Apples 2
Problem 8 Spill the Tea 3
Problem 9 Supermarket Search 4
Problem 10 Wardrobe 4
Problem 11 Party Hats 5

© NZOI - New Zealand Olympiad in Informatics

Days in a Leap Year

Problem 1 (5 points)

Explanation

There are normally 365 days in a year but on a leap year there is an extra day. If you didn't know during a contest, then guessing similar numbers around 365 is a valid strategy. There are no penalties for multiple submissions.

Example Solution 1

```
print(366)
```

Limited Addition

Problem 2 (5 points)

Explanation

Calling `input()` will get each line as a string. These must be converted to integers before you can add them. In the first example, the variables `a` and `b` are strings which are converted to `int` inside `print`. In the second example, they are converted to integers straight away.

Example Solution 1

```
a = input()
b = input()
print(int(a) + int(b))
```

Example Solution 2

```
a = int(input())
b = int(input())
print(a + b)
```

Stickers

Problem 3 (5 points)

Explanation

Even though the second line looks like a number, the program does not need to do any numeric operations on it so we can keep both inputs as strings and simply substitute them into the format string.

Example Solution 1

```
item = input()
price = input()
print(f"{item} are on special for ${price}!")
```

Example Solution 2

```
item = input()
price = input()
print(item, "are on special for", "$" + price + "!")
```


© NZOI - New Zealand Olympiad in Informatics

Tech Support

Problem 4 (5 points)

Explanation

Use `.split()` to separate the first line of input into two strings. Substitute these two strings into the output. Make sure there is no space between the full stop and the second string. There are multiple valid ways to do this. E.g. an f-string, string concatenation with `+`, and using `print` with `end=` or `sep=`.

Example Solution 1

```
line = input()
a, b = line.split()
print(f"Don't worry! Go to {a} and then find {b}.")
```

AI Chat-bot

Problem 5 (5 points)

Explanation

You may treat a string like a list of characters. We can use index `-1` to check the last character of the string. Alternatively, you could use index `len(string) - 1` to get the last character.

Example Solution 1

```
string = input()
if string[-1] == "?":
    print("Yes.")
else:
    print("Okay.")
```

Ribbons

Problem 6 (5 points)

Explanation

In Python you can do an **integer division** using `a // b`, which means divide `a` by `b` and round down to the nearest integer. Equivalently, how many whole `b` go into `a`.

You can get the **remainder** using `a % b`, which means what is left over after `a // b`.

Example Solution 1

```
a = int(input())
b = int(input())
print(a // b, a % b)
```

Alex's Apples

Problem 7 (10 points)

By Georgia

This problem has two parts. Submit each part to receive points.

Explanation

For both these solutions, it is also valid to use the modulus operator `%`. However, you then need to make sure you are handling the case where the inputs might be zero, as any number modulus zero will

© NZOI - New Zealand Olympiad in Informatics

give a divide by zero error. In the case of this problem, you are guaranteed **positive** integers as inputs, so you don't need to worry. Make sure you always read the question carefully. When this question was used in a contest it said **non-negative** integers instead, which means it includes zero. Lots of teams didn't get the points for this question because of it.

Part 1

Example Solution 1

```
total_apples = int(input())
box_size = int(input())
amount_of_boxes = int(input())
total_capacity = box_size * amount_of_boxes

if total_apples <= total_capacity:
    print("All apples safely stored.")
else:
    print("Apples not safely stored.")
```

Part2

Example Solution 1

```
total_apples = int(input())
box_size = int(input())
amount_of_boxes = int(input())
total_capacity = box_size * amount_of_boxes

if total_apples <= total_capacity:
    print("All apples safely stored.")
else:
    remainder = total_apples - total_capacity
    if remainder == 1:
        print("An apple is unsafe.")
    else:
        print(f"'{remainder}' apples are unsafe.")
```

Spill the Tea

Problem 8 (10 points)

Example Solution 1

```
N = int(input())

for d in range(1, N + 1):
    phrase = input()
    print(f'Day {d}: The phrase today is "{phrase}".')
```


© NZOI - New Zealand Olympiad in Informatics

Supermarket Search

Problem 9 (10 points)

Inspired by Sophie

Example Solution 1

```
found = False
line = input()
while line != "END":
    if line == "Honey":
        found = True
        break
    line = input()

if found:
    print("Sweet tea, full of glee.")
else:
    print("Sour tea, it cannot be!")
```

Wardrobe

Problem 10 (20 points)

Explanation

There are multiple ways to solve this. This solution uses two lists although it would be easier to use sets. The syntax `[0] * len(wardrobe)` will create a list that has all zeros and is the same length as the wardrobe list. Wardrobe is a list of strings. Every time an item in the wardrobe is found, the corresponding location in the `got` list is marked off by assigning `1` to it. If we `sum` all the `1`'s, then that tells us how many items we have. We can only do it because the problem states that wardrobe items are unique.

Example Solution 1

```
wardrobe = input().split()
got = [0] * len(wardrobe)

item = input()
i = 0
complete = None
while item != "STOP":
    if item in wardrobe:
        got[wardrobe.index(item)] = 1
        if sum(got) == len(wardrobe):
            complete = i
            break
    item = input()
    i += 1

if complete is None:
    print("It be like that.")
else:
    print(complete + 1)
```


© NZOI - New Zealand Olympiad in Informatics

Party Hats

Problem 11 (20 points)

Explaination

A helpful technique with any problem is to break it down into achievable steps.

For example:

1. Start by writing a program that prints the bottom row of the triangle. Eg. `* * *` for an input of 3.
2. Add the top row: a single star that is centred using space characters `" "`. Can we calculate how many spaces we need for a given size? Can we then construct a string of this many spaces and print it?
3. Each middle row contains two stars. Try printing just the left side of the triangle.
4. Finally, add the right side into the middle rows.

You will also need to think about the case when the size is 1. In this case, the answer is a single star. One way is to only print the bottom row when `size > 1`.

Example Solution 1

```
size = int(input())

# Top row
top = ""
for i in range(size - 1):
    top = top + " "
print(top + "*")

# Middle rows
for row in range(1, size - 1):

    # Leading spaces
    mid = ""
    for i in range(size - row - 1):
        mid = mid + " "

    # First star
    mid =  mid + "*"

    # Middle spaces
    for i in range(2 * row - 1):
        mid = mid + " "

    # Last star
    print(mid + "*")

# Bottom row
if size > 1:
    bot = ""
    for i in range(size):
        bot = bot + "* "
    print(bot)
```

Note that it is okay to print extra spaces at the end of a line. These will be ignored when comparing your answer to the correct answer.

© NZOI - New Zealand Olympiad in Informatics

Example Solution 2

```
size = int(input())
print(" " * (size - 1) + "*")
for i in range(1, size - 1):
    print(" " * (size - i - 1) + "*" + " " * (2 * i - 1) + "*")
if size > 1:
    print("* " * (size - 1) + "*")
```

This second solution uses string multiplication to do the same as the previous solution. There are often multiple correct ways to solve a problem.

© NZOI - New Zealand Olympiad in Informatics